M

MERCADO PULSE

LABORATORIO DE EVENTOS

GUÍA PRÁCTICA / STREAMING DE DATOS

Kafka conectado a una plataforma real

Manual para estudiantes basado en Mercado Pulse

QUÉ VAS A APRENDER

Levantar un broker local, seguir un pedido por cuatro tópicos, comprender productores y consumidores, inspeccionar mensajes y explicar cómo llega el estado al navegador.

Nivel: inicial a intermedio | Tiempo sugerido: 60-90 minutos

Proyecto local: Node.js + KafkaJS + Apache Kafka 4.1.2 + Server-Sent Events

Edición: octubre de 2026

CAPÍTULO 01 / MERCADO PULSE

El mapa antes de empezar

La idea esencial: la interfaz no habla directamente con Kafka. El servidor Node.js publica y consume eventos; luego envía el estado resultante al navegador.

¿Qué problema resuelve Kafka?

En una aplicación tradicional, una operación puede llamar directamente a muchas partes del sistema. Aquí cada cambio importante se publica como **evento**. Kafka almacena esos eventos en tópicos y permite que distintos procesos los lean a su ritmo. Un pedido puede activar pagos, inventario y despacho sin que el navegador tenga que coordinar cada paso.

Concepto	En este proyecto
Broker	El contenedor Docker que recibe y guarda los mensajes.
Tópico	Un flujo con nombre, por ejemplo commerce.orders.
Productor	Código que publica un evento mediante producer.send().
Consumidor	Código que lee eventos con consumer.run() y eachMessage.
Partición	Subdivisión ordenada de un tópico; cada tópico tiene 3.
Offset	Posición de un mensaje dentro de su partición.
Grupo de consumidores	Identidad que permite repartir trabajo y recordar progreso.

REGLA DE ORDEN

La clave de cada mensaje es orderId. Así, los mensajes de un mismo pedido van a la misma partición **dentro de cada tópico**. El orden no es global entre los cuatro tópicos.

Pista de lectura: primero sigue el recorrido de un pedido; después mira las funciones publish(), startConsumer() y project() en server.js.

CAPÍTULO 02 / MERCADO PULSE

Arquitectura de Mercado Pulse

El esquema resume las responsabilidades. Las flechas indican quién publica o consume información.

```
NAVEGADOR                    SERVIDOR NODE.JS                  KAFKA
Crear pedido  --POST-->  createOrder() --publish()--> commerce.orders
                                    order-worker <--------|
                         payment.evaluated ------------> commerce.payments
                         inventory.checked ------------> commerce.inventory
                               fulfillment-worker <-----|
                         fulfillment.dispatched -------> commerce.fulfillment
Panel <--- SSE /api/stream <--- project() <--- cuatro topicos
```

Las cuatro piezas del servidor

- **Administrador:** al arrancar consulta los tópicos existentes y crea los que faltan con 3 particiones y factor de replicación 1.
- **Productor:** publica pedidos y eventos derivados. Convierte el objeto JavaScript a JSON y usa orderId como clave Kafka.
- **Workers:** un grupo procesa pedidos y otro procesa inventario. Los grupos mantienen offsets propios.
- **Proyección:** un consumidor adicional lee los cuatro tópicos desde el comienzo y reconstruye un estado en memoria para el panel y el observatorio.

¿Por qué SSE?

El servidor envía actualizaciones por **GET /api/stream** con Content-Type **text/event-stream**. El navegador abre un EventSource y recibe cada instantánea. SSE es una conexión unidireccional del servidor al navegador; para crear pedidos se usa un POST normal.

> **DIFERENCIA IMPORTANTE**
> Kafka es el flujo entre componentes del servidor. SSE es el canal para actualizar la página web. El navegador no se conecta al puerto 9092.

CAPÍTULO 03 / MERCADO PULSE

Preparar y levantar el entorno

Ejecuta estos pasos desde la carpeta raíz del proyecto, en PowerShell.

Requisitos

- Node.js 20 o superior: comprueba con **node --version**.
- Docker Desktop iniciado: comprueba con **docker version**.
- Puertos locales 9092 (Kafka) y 3000 (plataforma) disponibles.

Paso 1 - iniciar el broker

```
docker compose up -d
docker compose ps
```

El archivo compose.yaml usa la imagen apache/kafka:4.1.2. Ejecuta un solo nodo en modo KRaft y anuncia localhost:9092 porque Node.js corre en tu equipo, fuera del contenedor. El volumen kafka-data conserva los mensajes entre reinicios.

Paso 2 - instalar y ejecutar la plataforma

```
npm install
npm start
```

Espera el mensaje **Kafka conectado: localhost:9092**. El servidor crea los tópicos si faltan y el simulador comienza a publicar pedidos automáticamente. Abre **http://localhost:3000**.

Paso 3 - comprobar la salud

```
Invoke-RestMethod http://localhost:3000/api/health
```

Debes ver **status: ok** y **kafka: true**. Si aparece connecting, espera unos segundos mientras Kafka termina de iniciar.

> **AL TERMINAR**
> Detén Node.js con Ctrl+C. Luego ejecuta docker compose down. Evita añadir -v si quieres conservar los eventos: esa opción elimina el volumen Kafka.

CAPÍTULO 04 / MERCADO PULSE

Seguir un pedido, evento por evento

Un pedido estándar suele producir cuatro eventos. Los casos de rechazo y falta de stock cortan el recorrido antes del despacho.

Orden	Tópico y evento	Qué representa
1	commerce.orders **order.created**	Un pedido nuevo con cliente, producto, monto y escenario.
2	commerce.payments **payment.evaluated**	Resultado approved o declined. Si se rechaza, el worker se detiene.
3	commerce.inventory **inventory.checked**	Resultado reserved u out_of_stock. Solo se publica si el pago fue aprobado.
4	commerce.fulfillment **fulfillment.dispatched**	Transportista y seguimiento. Solo ocurre con stock reservado.

La forma de un mensaje

```
{
  "eventId": "uuid-generado-por-el-servidor",
  "type": "order.created",
  "orderId": "MP-1234567-321",
  "occurredAt": "2026-10-28T12:00:00.000Z",
  "customer": "Camila Gonzalez",
  "product": "Audifonos Studio Pro",
  "total": 89990,
  "scenario": "normal"
}
```

Los campos comunes son eventId, type, orderId y occurredAt. El valor Kafka contiene JSON; la clave Kafka es orderId. Los eventos de pagos, inventario y despacho agregan sus propios campos. La partición y el offset son metadatos de Kafka que se ven en el observatorio, no parte del payload publicado.

EJEMPLO DE ESTADO

order.created → processing; pago aprobado + stock reservado → ready; despacho generado → dispatched. Un pago rechazado → failed. Sin stock → attention.

CAPÍTULO 05 / MERCADO PULSE

Dónde está la conexión en el código

Abre server.js para localizar los fragmentos mencionados en esta página.

1. Crear clientes

```
const kafka = new Kafka({ clientId: 'mercado-pulse',
  brokers: ['localhost:9092'] });
const admin = kafka.admin();
const producer = kafka.producer();
```

El proyecto obtiene brokers desde KAFKA_BROKERS; si no existe, usa localhost:9092. El administrador gestiona tópicos, el productor escribe y los consumidores leen.

2. Publicar un evento

```
await producer.send({
  topic,
  messages: [{ key: payload.orderId,
               value: JSON.stringify(payload) }]
});
```

3. Suscribirse y procesar

```
await consumer.subscribe({ topic, fromBeginning });
await consumer.run({ eachMessage: async ({ topic, partition, message }) => {
  const payload = JSON.parse(message.value.toString());
  await handler(topic, payload, partition, message.offset);
}});
```

El worker de pedidos usa el grupo **mercado-pulse-order-worker**. El de inventario usa **mercado-pulse-fulfillment-worker**. El lector del panel usa un grupo nuevo en cada arranque y fromBeginning=true para reconstruir la vista desde los mensajes almacenados.

4. Convertir eventos en pantalla

project() actualiza un Map de pedidos, los conteos y los intervalos del gráfico. broadcast() envía una instantánea JSON a todos los clientes SSE. app.js y kafka.js reciben el stream con EventSource y actualizan sus interfaces.

CAPÍTULO 06 / MERCADO PULSE

Práctica guiada y observatorio

Usa la plataforma para ver la diferencia entre el flujo de negocio y los mensajes reales.

Actividad A - pedido normal

- Abre **http://localhost:3000** y selecciona **Crear pedido**. Elige Pedido estándar.
- Anota su orderId y observa cómo cambia de Procesando a Preparando y, normalmente, a Enviado.
- Abre **http://localhost:3000/__kafka**. Filtra por tópico y busca ese orderId en el stream.
- Pulsa **Ver JSON** para inspeccionar el payload, la partición y el offset.

Actividad B - falta de stock

```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:3000/api/orders `
  -ContentType 'application/json' `
  -Body '{"scenario":"stockout"}'
```

Este escenario aprueba el pago, publica inventory.checked con out_of_stock y no publica fulfillment.dispatched para ese pedido. Busca el estado **Sin stock** en la plataforma.

Actividad C - mirar Kafka desde su CLI

```
docker exec mercado-pulse-kafka /opt/kafka/bin/kafka-topics.sh `
  --bootstrap-server localhost:9092 --list

docker exec mercado-pulse-kafka /opt/kafka/bin/kafka-console-consumer.sh `
  --bootstrap-server localhost:9092 `
  --topic commerce.orders --from-beginning `
  --max-messages 3
```

PREGUNTAS PARA DISCUTIR

¿Por qué el offset 4 de payments no corresponde necesariamente al offset 4 de orders? ¿Qué cambia al detener el simulador? ¿Por qué el visor puede reconstruir datos tras reiniciar Node.js?

CAPÍTULO 07 / MERCADO PULSE

Diagnóstico, límites y siguientes pasos

Usa esta página cuando algo no responda como esperas.

Síntoma	Comprobación y solución
El panel muestra "Conectando"	Comprueba docker compose ps y GET /api/health. Inicia Docker Desktop y espera a que Kafka esté listo.
Error en puerto 9092 o 3000	Otro programa usa el puerto. Libéralo o cambia PORT para Node.js; si cambias el puerto de Kafka, ajusta también listeners y KAFKA_BROKERS.
No aparecen eventos nuevos	Comprueba que el botón diga "Pausar simulación". Crea un pedido manual y observa la salida de npm start.
Reinicio con datos antiguos	Es esperado: Kafka conserva mensajes en kafka-data y la vista relee los tópicos. docker compose down -v borra los datos de la demo.
El visor no abre	Visita la ruta exacta /__kafka en el mismo puerto de la plataforma; no figura en el menú principal.

Límites deliberados de esta demo

- Un solo broker y factor de replicación 1: adecuado para aprendizaje local, sin alta disponibilidad.
- Pagos, stock, clientes y transportistas son simulados. No hay transacciones reales.
- La proyección está en memoria y se recompone al iniciar; no es una base de datos de producción.
- El procesamiento puede repetir un mensaje después de un fallo antes de guardar el offset. Para producción se necesitaría idempotencia y manejo de errores más completo.
- La ruta /__kafka es poco visible, pero no está protegida por autenticación.

Para profundizar

Estudia retención, escalado de consumidores, esquemas de eventos, idempotencia, claves y particiones. Como extensión, agrega un quinto tópico para devoluciones o crea alertas cuando aumente la tasa de rechazos.

CAPÍTULO 08 / MERCADO PULSE

Glosario y fuentes

Una hoja de repaso para explicar el proyecto con tus propias palabras.

Término	Definición breve
Evento	Hecho de negocio ocurrido en un momento concreto.
Clave	Dato usado para seleccionar la partición; aquí, orderId.
Offset	Número de posición de un mensaje dentro de una partición.
Consumer group	Conjunto lógico de lectores que comparte trabajo y progreso.
Proyección	Vista calculada a partir de eventos; aquí vive en memoria.
SSE	Flujo HTTP del servidor al navegador para actualizar la interfaz.
KRaft	Modo de coordinación de Kafka usado por el broker local.

Resumen en una frase

EL RECORRIDO COMPLETO

Una acción crea un evento → Kafka lo almacena en un tópico → un worker lo procesa y publica más eventos → otro consumidor forma el estado → SSE lo lleva al navegador.

Fuentes para continuar

- Apache Kafka, introducción: kafka.apache.org/intro/
- Apache Kafka 4.1, inicio rápido: kafka.apache.org/41/getting-started/quickstart/
- KafkaJS, producción y consumo: kafka.js.org/docs/producing y kafka.js.org/docs/consuming
- MDN, Server-Sent Events: guía de SSE en MDN

Archivo base de este manual: README.md, compose.yaml, package.json y server.js del proyecto Mercado Pulse.